Общие требования:

- **исходный код должен** быть оформлен в соответствии с (https://habr.com/ru/post/172091/) или (https://wiki.qt.io/Qt_Coding_Style).

- исходный код должен быть размещён в одном из удалённых репозиториев (GitHub, BitBucket и т.п.) - программа не должна иметь утечек памяти и/или падений вследствие необработанных исключений.

В качестве доказательства необходимо подготовить тестовое покрытие для программы
(в виде текстового списка: **входные данные/действия** - реакция программы).

Организация удалённого репозитория: - основная разработка ведётся в ветке **develop**, допускается создание дополнительных веток, если это необходимо. Допускается ветвление от **develop**. Созданная ветвь соответствует решению некоторй подзадачи (**taski**, ...), которая после доработки и тестирования сливается в **develop**. Готовое задание сливается в ветку **master**. Проверка задания выполняется с ветки **master**. - сообщения коммитов отражают совершённые в коммите изменения. Детальное описание размещается в комментарии к коммиту. Недопустимы односложные/неинформативные/содержащие множественные опечатки коммиты (примеры: "fix", "final", "вроде всё" и т.п.). - время между коммитами должно быть соразмерно объёму выполненных работ. - в ветку **master** сливаются только готовые версии программы.

Для каждой лабораторной работы необходимо подготовить ответ для защиты, в соответствии со следующей структурой (*отражаем в readme*).

1. Постановка задачи

Описание постановки задачи.

2. Предлагаемое решение

Предлагаемое решение излагается на уровне идеи.

Описывается общее решение задачи, с указанием структуры решения, в частности описание предлагаемых сущностей(классы), функций.

Архитектура ПО описывается и представляется с помощью UML диаграммы. Если предложенная архитектура ПО базируется на одном их известных паттернов, также необходимо рассказать назначение и архитектуру паттерна. При использовании в решении готовых технологий необходимо продемонстрировать полное понимание технологии, в частности

- суть технологии;
- какую задачу решает используемая технология;
- какие шаги применялись для использования рассматриваемой технологии;

Уметь объяснять конструкции языка C++, а также используемые технологии при разработки ПО, поведение ПО, подходы безопасной разработки.

3. Инструкция пользователя

В этом разделе демонстрируются варианты взаимодействия с вашим ПО с точки зрения пользователя. Если требуется, то предоставляете готовые примеры (входные файлы с исходными данными и т.д.).

4. Тестирование

В этом разделе описывается процесс тестирования, в частности все случаи (case1, case2) допустимые и недопустимые при которых ваше ПО выдает адекватную реакцию. Также, тестовые примеры, относительно которых проводилось тестирование.